

Q) Explain P, NP, NP-Hard, and NP-Complete problems with examples. [7 Marks]
(SPPU BE DAA)

Answer

Introduction

In **Complexity Theory**, computational problems are classified based on the time required to solve or verify them. The four important classes are:

- P Problems
 - NP Problems
 - NP-Complete Problems
 - NP-Hard Problems
-

1. P (Polynomial Time) Problems

Definition

P-Class consists of problems that can be **solved in polynomial time** using a **deterministic algorithm**.

Time Complexity: $O(1)$, $O(n)$, $O(n^2)$, $O(n^3)$, $O(n \log n)$

Examples

- Binary Search
 - Merge Sort
 - Linear Search
 - Finding Minimum Element
-

2. NP (Non-deterministic Polynomial Time) Problems

Definition

NP-Class consists of problems whose **solution can be verified in polynomial time**, even though finding the solution may be difficult.

Examples

- Boolean SAT
- Hamiltonian Cycle (Decision Problem)

- Sudoku Verification

3. NP-Complete Problems

Definition

A problem is **NP-Complete** if:

1. It belongs to **NP**.
2. Every NP problem can be reduced to it in polynomial time.

These are the **hardest problems in NP**.

Examples

- 3-SAT
- Vertex Cover
- Clique Problem
- Subset Sum

4. NP-Hard Problems

Definition

NP-Hard problems are at least as difficult as NP problems. They **may or may not belong to NP**, and their solutions **cannot always be verified in polynomial time**.

Examples

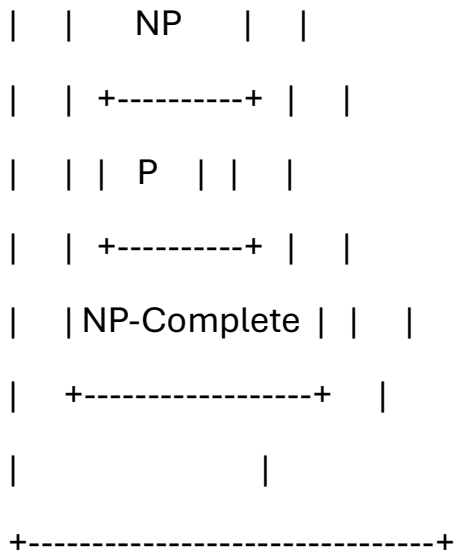
- Travelling Salesman Problem (Optimization)
- Hamiltonian Cycle (Optimization)
- Job Scheduling Optimization

Relationship Diagram

NP-Hard

+-----+

| |
| +-----+ |



Difference Between P, NP, NP-Complete and NP-Hard

Class	Meaning	Solved in Polynomial Time	Verified in Polynomial Time	Examples
P	Easy problems	Yes	Yes	Binary Search, Merge Sort
NP	Solution can be verified quickly	Not necessarily	Yes	SAT, Hamiltonian Cycle
NP-Complete	Hardest problems in NP	No known polynomial algorithm	Yes	3-SAT, Vertex Cover
NP-Hard	At least as hard as NP	Not necessarily	Not necessarily	TSP (Optimization), Hamiltonian Cycle (Optimization)

Conclusion

- **P** problems are easy to solve.
- **NP** problems are easy to verify.
- **NP-Complete** problems are the hardest problems in NP.

- **NP-Hard** problems are at least as difficult as NP problems and may not belong to NP.

Q) Briefly explain P and NP problems in the context of Complexity Theory, with examples. [8 Marks] (SPPU BE DAA)

Answer

Introduction

Complexity Theory is the study of how much **time** and **memory** an algorithm requires to solve a problem. Based on their computational complexity, problems are classified into different classes. The two basic classes are **P** and **NP**.

1. P (Polynomial Time) Problems

Definition

P-Class consists of problems that can be **solved in polynomial time** by a **deterministic algorithm**.

The running time is generally:

$$O(1), O(n), O(n^2), O(n^3), O(n \log n)$$

These problems are considered **easy and efficiently solvable**.

Characteristics

- Solved in polynomial time.
- Uses deterministic algorithms.
- Easy to solve and verify.
- Efficient for practical applications.

Examples

- Binary Search
- Linear Search
- Merge Sort

- Finding Minimum Element
 - Matrix Multiplication
-

2. NP (Non-deterministic Polynomial Time) Problems

Definition

NP-Class consists of problems whose **solution can be verified in polynomial time** by a deterministic algorithm, although finding the solution may be difficult.

Characteristics

- Solution is difficult to find.
- Solution can be verified quickly.
- Every **P problem is also an NP problem**.

Examples

- Boolean SAT
 - Hamiltonian Cycle (Decision Problem)
 - Travelling Salesman Problem (Decision Version)
 - Sudoku Verification
-

Difference Between P and NP

P Problems	NP Problems
Solved in polynomial time	Solution verified in polynomial time
Easy to solve	Difficult to solve but easy to verify
Uses deterministic algorithms	Solved using non-deterministic concept
Every P problem belongs to NP	NP may contain problems outside P
Example: Merge Sort, Binary Search	Example: SAT, Hamiltonian Cycle

Relationship Diagram

NP

```

+-----+
|  +-----+  |
|  | P |  |
|  +-----+  |
+-----+

```

Explanation:

- **$P \subseteq NP$** , i.e., every P problem is also an NP problem.
 - It is still **unknown whether $P = NP$** .
-

Conclusion

- **P problems** are easy to solve and verify in polynomial time.
- **NP problems** may be difficult to solve, but their solutions can be verified efficiently.
- Understanding **P** and **NP** is important in complexity theory for designing efficient algorithms and studying difficult computational problems.

Q4) What is an NP-Complete class problem? How would you prove the Vertex Cover problem is NP-Complete? [8 Marks] (SPPU BE DAA)

Answer

1. NP-Complete Problem

Definition

A problem is called **NP-Complete** if it satisfies the following two conditions:

1. **The problem belongs to NP**, i.e., its solution can be verified in polynomial time.
2. **Every NP problem can be reduced to it in polynomial time.**

Thus, NP-Complete problems are the **hardest problems in NP**.

Examples

- 3-SAT
- Vertex Cover

- Clique
 - Subset Sum
-

2. Vertex Cover Problem

Definition

A **Vertex Cover** of a graph $G(V,E)$ is a set of vertices such that **every edge in the graph is incident to at least one selected vertex**.

The problem is:

Given a graph $G(V,E)$ and an integer k , determine whether there exists a vertex cover of size $\leq k$.

Example

A

/ \

B---C

\ /

D

Edges:

AB, AC, BC, BD, CD

Choose vertices:

{B, C}

These two vertices cover all edges:

- AB ✓
- AC ✓
- BC ✓
- BD ✓
- CD ✓

Hence,

Vertex Cover = {B, C}

$k = 2$

3. Proof that Vertex Cover is NP-Complete

To prove that **Vertex Cover** is NP-Complete, we must prove two conditions.

Step 1: Vertex Cover belongs to NP

- Suppose a set of vertices is given.
- We can check whether every edge is covered by at least one selected vertex.
- This verification takes **polynomial time**.

Therefore,

Vertex Cover $\in$ NP

Step 2: Polynomial-Time Reduction

- The **3-SAT problem** is already known to be **NP-Complete**.
- A polynomial-time reduction can be made from **3-SAT $\rightarrow$ Vertex Cover**.
- This means any instance of 3-SAT can be transformed into an equivalent Vertex Cover instance in polynomial time.

Therefore,

3-SAT $\leq_p$ Vertex Cover

Since:

- Vertex Cover is in NP.
- Every NP problem (through 3-SAT) can be reduced to Vertex Cover in polynomial time.

Hence,

Vertex Cover is NP-Complete.

Flow Diagram of Proof

3-SAT (NP-Complete)

|

Polynomial-Time Reduction



Vertex Cover

|

Verification in Polynomial Time



Vertex Cover $\in$ NP

$\therefore$ Vertex Cover is NP-Complete

Conclusion

The **Vertex Cover** problem satisfies both conditions of an NP-Complete problem:

1. It belongs to **NP** because a proposed solution can be verified in polynomial time.
2. It can be obtained by **polynomial-time reduction from 3-SAT**, an NP-Complete problem.

Hence, **Vertex Cover is an NP-Complete problem.**

**What do you understand by NP-Complete and NP-Hard problems? Give examples.
[6 Marks] (SPPU BE DAA)**

Answer

Introduction

In **Complexity Theory**, some computational problems are very difficult to solve. These problems are classified into **NP-Complete** and **NP-Hard** based on their complexity.

1. NP-Complete Problems

Definition

A problem is called **NP-Complete** if it satisfies the following conditions:

1. The problem belongs to **NP**, i.e., its solution can be **verified in polynomial time**.
2. Every NP problem can be **reduced to it in polynomial time**.

NP-Complete problems are the **hardest problems in the NP class**.

Examples

- 3-SAT
 - Vertex Cover
 - Clique Problem
 - Subset Sum
-

2. NP-Hard Problems

Definition

A problem is called **NP-Hard** if it is **at least as hard as every problem in NP**.

- It **may or may not belong to NP**.
- Its solution **cannot always be verified in polynomial time**.

Examples

- **Hamiltonian Cycle** (*As mentioned in the SPPU syllabus*)
 - Travelling Salesman Problem (Optimization)
 - Job Scheduling Optimization
-

Difference between NP-Complete and NP-Hard

NP-Complete	NP-Hard
Belongs to NP	May or may not belong to NP
Solution can be verified in polynomial time	Solution may not be verified in polynomial time
Every NP problem can be reduced to it	At least as hard as NP problems

NP-Complete

Examples: 3-SAT, Vertex Cover

NP-Hard

Examples: Hamiltonian Cycle, TSP (Optimization)

Hamiltonian Cycle (NP-Hard)

Definition

A **Hamiltonian Cycle** is a cycle in a graph that **visits every vertex exactly once and returns to the starting vertex**.

Example

A ----- B

| |

| |

D ----- C

One Hamiltonian Cycle is:

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$

Finding such a cycle for an arbitrary graph is computationally difficult. Therefore, the **Hamiltonian Cycle problem** is considered an **NP-Hard problem** in the context of the SPPU syllabus.

Conclusion

- **NP-Complete** problems are the hardest problems **within NP** and their solutions can be verified in polynomial time.
- **NP-Hard** problems are at least as difficult as NP problems and may not belong to NP.
- **Vertex Cover** and **3-SAT** are NP-Complete problems, whereas **Hamiltonian Cycle** is the NP-Hard example specified in the **SPPU BE DAA syllabus**.